

VSSearchInput — Setup Variants Reference

Practical code snippets showing common setup variants supported by VSSearchInput. These examples can be copied directly into projects.

1. Minimal Setup (Simple API)

```
VSSearchInput.init(input, {  
  api: '/api/search-driver'  
});
```

2. Custom search key

```
VSSearchInput.init(input, {  
  api: '/api/search-driver',  
  searchTerm: 'q'  
});
```

3. Input autocomplete mode

```
VSSearchInput.init(input, {  
  type: 'input',  
  api: '/api/search-driver'  
});
```

4. Select dropdown mode

```
VSSearchInput.init(input, {  
  type: 'select',  
  api: '/api/search-driver'  
});
```

5. Select mode with prefetch

```
VSSearchInput.init(input, {  
  type: 'select',  
  api: '/api/search-driver',  
  prefetch: true  
});
```

6. Limit dropdown height

```
VSSearchInput.init(input, {  
  api: '/api/search-driver',  
  maxDropdownHeight: '380px'  
});
```

7. Table / Column rendering

```

VSSearchInput.init(input, {
  api: '/api/search-driver',
  type: 'select',

  columns: {
    code: 'ID',
    name: 'Driver',
    phone_number: 'Phone'
  },

  showColumnHeader: true
});

```

8. Advanced column mapping

```

columns: {
  code: { label: 'ID', field: 'code' },
  name: { label: 'Driver Name', field: 'name' },
  phone: { label: 'Phone', field: 'phone_number' }
}

```

9. Handling selection

```

VSSearchInput.init(driverInput, {
  api: '/api/search-driver',

  onSelect: item => {
    driverInput.value = `${item.name} (${item.code})`;
    driverInput.dataset.driverId = item.id;
  }
});

```

10. Passing static parameters

```

api: {
  endpoint: '/api/search-driver',
  params: {
    branch_id: 5
  }
}

```

11. Passing dynamic parameters

```

api: {
  endpoint: '/api/search-driver',
  params: inst => ({
    branch_id: currentBranchId
  })
}

```

12. Using XQuery backend

```

VSSearchInput.init(input, {
  query: {
    from: 'drivers',

```

```

        select: ['id', 'code', 'name'],
        limit: 20,
        searchFields: {
            name: 'LIKE',
            code: 'LIKE'
        }
    }
});

```

13. Custom response processing

```

processResponse: res =>
    res.data.map(x => ({
        label: x.name,
        id: x.id
    })))

```

14. Custom dropdown rendering

```

onFetchComplete(res, dropdown, inst) {
    dropdown.innerHTML = customHtml;
}

```

15. Prefetch select with columns (common setup)

```

VSSearchInput.init(driverInput, {
    type: 'select',
    api: '/api/search-driver',
    prefetch: true,
    maxDropdownHeight: '380px',

    columns: {
        code: 'ID',
        name: 'Driver',
        phone_number: 'Phone'
    },

    showColumnHeader: true,

    onSelect: item => {
        driverInput.value = `${item.name} (${item.code})`;
        driverInput.dataset.driverId = item.id;
    }
});

```

16. Reset when modal opens

```

instance.reset();

```